

HOMework 1

DECISION TREES AND LINEAR SEPARATORS¹

CMU 10-701: INTRODUCTION TO MACHINE LEARNING (FALL 2024)

<https://piazza.com/cmu/fall2024/10701/home>

OUT: Wednesday, Sept 4th, 2024

DUE: Wednesday, Sept 18th, 2024, 11:59pm

START HERE: Instructions

- **Collaboration policy:** Collaboration on solving the homework is allowed, after you have thought about the problems on your own. It is also OK to get clarification (but not solutions) from books or online resources, again after you have thought about the problems on your own. There are two requirements: first, cite your collaborators fully and completely (e.g., “Jane explained to me what is asked in Question 2.1”). Second, write your solution *independently*: close the book and all of your notes, and send collaborators out of the room, so that the solution comes from you only. See the Academic Integrity Section in our [Course Syllabus](#) for more information:
 - **Late Submission Policy:** See the late homework policy in our [Course Syllabus](#).
 - **Submitting your work:**
 - **Gradescope:** Submit your homework to the Homework 1: Written Gradescope submission slot. Please use the provided template. The best way to format your homework is by using the Latex template released in the handout and writing your solutions in Latex. However submissions can be handwritten onto the template, but should be labeled and clearly legible. If your writing is not legible, you will not be awarded marks. Each derivation/proof should be completed in the boxes provided below the question, **you should not move or change the sizes of these boxes** as Gradescope is expecting your solved homework PDF to match the template on Gradescope. If you find you need more space than the box provides you should consider cutting your solution down to its relevant parts, if you see no way to do this, please add an additional page at the end of the homework and guide us there with a ‘See page xx for the rest of the solution’.
- Regrade requests can be made after the homework grades are released, however this gives the TA the opportunity to regrade your entire paper, meaning if additional mistakes are found then points will be deducted.

For multiple choice or select all that apply questions, shade in the box or circle in the template document corresponding to the correct answer(s) for each of the questions. For L^AT_EX users, use ■ and ● for shaded boxes and circles, and don’t change anything else. If an answer box is included for showing work, **you must show your work!**

¹Compiled on Saturday 21st September, 2024 at 01:29

1 Decision Trees [40 pts]

Recall that the ID3 algorithm iteratively grows a decision tree from the root downwards. On each iteration, the algorithm replaces one leaf node with an internal node that splits the data based on one decision attribute (or feature). In particular, the ID3 algorithm chooses the split that reduces the entropy the most, but there are other choices. For example, since our goal in the end is to have the lowest error, why not instead choose the split that reduces error the most? In this problem, we will explore one reason why reducing entropy is a better criterion.

Consider the following simple setting. Let us suppose each example is described by n boolean features: $X = \langle X_1, \dots, X_n \rangle$, where $X_i \in \{0, 1\}$, and where $n \geq 5$. Furthermore, the target function to be learned is $f : X \rightarrow Y$, where $Y = X_1 \vee X_2 \vee \neg X_4 \vee X_5$. That is, $Y = 1$ if $X_1 = 1$ or $X_2 = 1$ or $X_4 = 0$ or $X_5 = 1$, and $Y = 0$ otherwise. Suppose that your training data contains all of the 2^n possible examples, each labeled by f . For example, when $n = 5$, a portion of the data set would look like

X_1	X_2	X_3	X_4	X_5	Y
0	0	0	0	0	1
1	0	0	0	0	1
0	1	0	0	0	1
1	1	0	0	0	1
0	0	1	0	0	1
1	0	1	0	0	1
0	1	1	0	0	1
1	1	1	0	0	1
0	0	0	1	0	0

1. [6 Points] How many mistakes does the best 1-leaf decision tree make, over the 2^n training examples? (The 1-leaf decision tree does not split the data even once)

Since $Y = X_1 \vee X_2 \vee \neg X_4 \vee X_5$, then Y only depends on the features X_1, X_2, X_4 , and X_5 . For the example where $n = 5$, we see that the majority class is $Y = 1$. The minority class only occurs in the following instances, in which X_3 is a free variable:

X_1	X_2	X_3	X_4	X_5	Y
0	0	0	1	0	0
0	0	1	1	0	0

The best decision tree is the one that always predicts the majority class. This tree will make only $2 = 2^1$ mistakes out of 2^5 possible feature combinations.

In the general case with n features, the tree will only make a mistake when the following 4 conditions, $X_1 = 0, X_2 = 0, X_4 = 1$, and $X_5 = 0$, are met. The remaining $n - 4$ features, $X_3, X_6, X_7, \dots, X_n$, are free variables that can take on any value (0 or 1). So, out of 2^n possible feature combinations, the number of instances where a mistake will be made, i.e., the prediction is $Y = 0$, would be 2^{n-4} .

Answer: 2^{n-4} mistakes

2. [8 Points] Is there a split that reduces the number of mistakes by at least one? (I.e., is there a decision tree with 1 internal node with fewer mistakes than your answer to part 1?) Why or why not?

There exists no split that reduces the number of mistakes. We can see this through examination of possible splits. Recall that Y only depends on the features X_1, X_2, X_4 , and X_5 , so we should see what happens when splitting on each of these features.

For X_1 :

Splitting on X_1 implies two cases: $X_1 = 0$ or $X_1 = 1$. When $X_1 = 1$, this is the positive case, so $Y = 1$ and there are no mistakes. However, when $X_1 = 0$, the output of the model is still conditional upon the features X_2, X_4 , and X_5 . When $X_2 = 0, X_4 = 1$, and $X_5 = 0$, then $Y = 0$, and the same mistakes (prior to splitting) will still occur. Thus, splitting on X_1 does not reduce the number of mistakes.

For X_2 :

Similarly to splitting on X_1 , we have either $X_2 = 1$ or $X_2 = 0$. For $X_2 = 1$, then $Y = 1$ and no mistakes occur. When $X_2 = 0$, the same errors occur when $X_1 = 0, X_4 = 1$, and $X_5 = 0$.

For X_4 :

We have 2 cases: 1) $X_4 = 0$, then $Y = 1$, and there are no mistakes. 2) $X_4 = 1$, then Y depends on X_1, X_2 , and X_5 , and the same mistakes will occur when $X_1 = 0, X_2 = 0$, and $X_5 = 0$.

For X_5 : Same reasoning as X_1, X_2 . The number of mistakes is not reduced.

Answer: no

3. [4 Points] What is the entropy of the output label Y for the 1-leaf decision tree (no splits at all)? You do not need to show your working. Please give your answer to 4 decimal places.

$$H(Y) = 0.3373$$

4. [4 Points] Is there a split that reduces the entropy of the output Y by a non-zero amount? If so, what is it, and what is the resulting conditional entropy of Y given this split?

We can try to split on one of the key features (X_1, X_2, X_4 or X_5), since these are the features that Y depends on.

If we split on X_1 , there are two cases: $X_1 = 1$ or $X_1 = 0$.

When $X_1 = 1$, then $Y = 1$ with certainty and the entropy is 0. In the other case, when $X_1 = 0$, then the output Y is conditional on $X_2 \vee \neg X_4 \vee X_5$.

The resulting conditional entropy is $H(Y|X_1 = 0)$, which will be less than the original entropy $H(Y) = 0.3373$ due to the reduced uncertainty in X_1 .

Answer: yes, split on X_1 , conditional entropy = $H(Y|X_1 = 0)$

5. [8 Points] Given any arbitrary target function $f : X \rightarrow Y$, what is the upper and lower bound of the entropy of the output label Y for a 1-leaf decision tree, i.e., what is the entropy of the distribution of target function f ? Please justify your answer.

For the 1-leaf decision tree, the lower bound of the entropy, denoted $H(Y)$, of the output label Y is $H(Y) = 0$. This is the case in which there is no randomness, and the output is completely deterministic. This occurs when the model predicts one class, all $Y = 0$ or all $Y = 1$. In this case, we calculate the entropy:

$$H(Y) = -p_1 \log_2(p_1) - p_0 \log_2(p_0),$$

where p_1 is the proportion of positive examples and p_0 is the proportion of negative examples. When only one class is predicted, both terms will converge to 0. To illustrate: For the $Y = 0$ case:

$$H(Y) = 0 \cdot \log_2(0) - 1 \cdot \log_2(1) = 0 - 0 = 0$$

For the $Y = 1$ case:

$$H(Y) = 1 \cdot \log_2(1) - 0 \cdot \log_2(0) = 0 - 0 = 0$$

Thus, the lower bound for the entropy is $H(Y) = 0$, and this occurs when $f : X \rightarrow Y$ outputs the same label for all samples.

The upper bound of the entropy of the output label is $H(Y) = 1$, which is the case where the output Y is completely uncertain. The maximum uncertainty occurs when the output labels are evenly split, and we can calculate the corresponding maximum entropy:

$$H(Y) = -0.5 \log_2(0.5) - 0.5 \log_2(0.5) = 0.5 + 0.5 = 1$$

Thus, the upper bound for the entropy is $H(Y) = 1$.

Answer: [0,1]

6. [10 Points] Consider the dataset in Table 1 for this problem. Given the four attributes on the left, we want to predict if the student got an A in the course. Feel free to use any mixture of code or solving-by-hand to solve the follow parts — we believe solving it by hand would be less work.

Feature 1	Feature 2	Feature 3	Feature 4	Outcome
0	0	1	1	1
1	1	0	1	0
0	0	1	1	1
1	1	1	1	0
1	0	0	0	0
0	0	0	0	1
1	0	0	0	1
1	0	1	0	0
1	1	0	1	1
1	1	0	0	0

Table 1: Features and Labels

Create a decision tree of depth 2 using the ID3 algorithm from class. The tree should have the same structure as the diagram in Figure 1. Note that 0 leads to the left branch and 1 leads to the right branch.

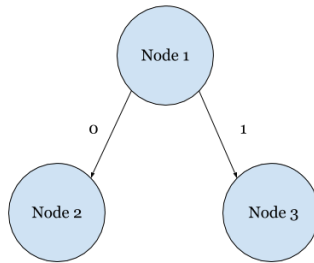


Figure 1: Decision tree structure for question 2

- (i) **[4 Points]** What is the attribute at Node 1? What is the information gain of this attribute? You do not need to show your work. (4 decimal places)

Attribute to split on: Feature 1

Information Gain: 0.3958

- (ii) **[3 Points]** What is the attribute at Node 2? What is the information gain of this attribute? You do not need to show your work. (4 decimal places)

Attribute to split on: N/A (pure node) ,

Information Gain: 0

- (iii) **[3 Points]** What is the attribute at Node 3? What is the information gain of this attribute? You do not need to show your work. (4 decimal places)

Attribute to split on: Feature 3

Information Gain: 0.4853

2 Problem 2: Accuracy of Perceptron Algorithm [30 Pts]

1. [15 Points] Lets suppose we have d weather models $\{M_i\}_{i=1}^d$, each of which predicts if it is going to rain the next day. Each model outputs either -1 or $+1$, where -1 corresponds to no rain and $+1$ corresponds to rain. We do not know anything about these models (some of these could be malicious). We only know that there is some subset of K of the d models whose majority vote always gives the correct prediction (where K is an odd number). However we dont know the identity of these K models.

Our goal is to learn the best possible way to combine the outputs of these d models to reliably predict rain. To be more formal, we would like to learn a function $h : \{-1, +1\}^d \rightarrow \{-1, +1\}$, which takes the predictions of the d models as inputs and reliably predicts whether it is going to rain the next day.

Suppose we use the Perceptron algorithm to learn a linear predictor h . Each day, we make a prediction based on the current h and update h the next day if our prediction was wrong. Show that you will make at most Kd wrong predictions using this approach.

The output (y_t) belongs to the set $\{-1, +1\}$, so let us generalize the update step of the perceptron algorithm as $w_{t+1} = w_t + y_t x_t$, with initial weight vector w_1 as the zero vector.

We are given the information that there is a subset K of the d models that always give the correct prediction through majority vote; thus, let us denote some ideal weight vector, w^* , such that $w^* \cdot x_t = y_t$.

Let us consider the quantities $w_{t+1} \cdot w^*$ and $\|w_{t+1}\|^2$.

First, $w_{t+1} \cdot w^*$:

$$w_{t+1} \cdot w^* = (w_t + y_t x_t) \cdot w^* = w_t \cdot w^* + y_t (w^* \cdot x_t)$$

Since $w^* \cdot x_t = y_t$, then:

$$w_{t+1} \cdot w^* = w_t \cdot w^* + y_t^2$$

Since $y_t \in \{-1, 1\}$, then $y_t^2 = 1$ and thus $w_{t+1} \cdot w^* = w_t \cdot w^* + 1$ So, each time a mistake is made, the dot product is increased by 1.

After M mistakes, the initial weight vector is updated by 1 for M times:

$$w_{M+1} \cdot w^* \geq w_1 \cdot w^* + M \implies w_{M+1} \cdot w^* \geq M$$

Now, consider $\|w_{t+1}\|^2$:

$$\|w_{t+1}\|^2 = \|w_t + y_t x_t\|^2 = \|w_t\|^2 + 2y_t(w_t \cdot x_t) + y_t^2 \|x_t\|^2$$

Since $y_t \in \{-1, 1\}$, then $y_t^2 = 1$, and since $x_t \in \{-1, 1\}^d$, then $\|x_t\|^2 = d$. Now, we have:

$$\|w_{t+1}\|^2 = \|w_t\|^2 + 2y_t(w_t \cdot x_t) + d$$

On a mistake, the middle term $2y_t(w_t \cdot x_t)$ is negative or 0, since an update is only made on a mistake. So:

$$\|w_{t+1}\|^2 \leq \|w_t\|^2 + d$$

After M mistakes, the norm of w_{M+1} is bounded: $\|w_{M+1}\|^2 \leq Md \implies \|w_{M+1}\| \leq \sqrt{Md}$

We have that: (1) $w_{M+1} \cdot w^* \geq M$ and (2) $\|w_{M+1}\| \leq \sqrt{Md}$

Additionally, w^* is a unit vector, so $w_{M+1} \cdot w^* \leq \|w_{M+1}\|$ (Cauchy-Schwarz inequality). Combining this with the previously derived facts, we get:

$$M \leq w_{M+1} \cdot w^* \leq \|w_{M+1}\| \leq \sqrt{Md} \implies M \leq \sqrt{Md} \implies \sqrt{M} \leq \sqrt{d} \implies M \leq d$$

Since we denoted w^* as the ideal weight vector for the correct predictions of K models, and the perceptron makes at most d mistakes per model (as shown above), then **the total number of mistakes, M , is bounded by Kd .**

2. **[15 Points]** In this sub problem we will consider a variant of the online Perceptron algorithm we saw in the class. In each iteration of the algorithm, given example X with label $y \in \{-1, +1\}$, we always update w as:

$$w \leftarrow w + yX$$

(Note that in the original perceptron algorithm, we update w only if the predicted label differs from the true label.)

Suppose the data has margin γ and all points lie inside a ball of radius R . Is there an upper bound on the number of mistakes the above algorithm can make? If yes, provide one. If not, provide an example where it can make infinitely many mistakes.

Let us denote w^* as the max-margin separator with $\|w^*\| = 1$ and denote γ as the margin. By definition of the margin, we know that $y(X \cdot w^*) \geq \gamma$ for each training example X with label $y \in \{-1, 1\}$. We initialize the weight vector w_1 as the zero vector and update $w_{t+1} = w_t + yX$ with each iteration.

Now, let us consider the quantities $w_{t+1} \cdot w^*$ and $\|w_{t+1}\|^2$.

First, $w_{t+1} \cdot w^*$:

$$w_{t+1} \cdot w^* = (w_t + yX) \cdot w^* = w_t \cdot w^* + y(w^* \cdot X)$$

We know $y(X \cdot w^*) \geq \gamma$, so

$$w_{t+1} \cdot w^* \geq w_t \cdot w^* + \gamma$$

So, after each mistake, the dot product $w_{t+1} \cdot w^*$ increases by at least γ .

After M mistakes, we update the initial dot product ($w_1 \cdot w^*$) by γ for M times:

$$w_{M+1} \cdot w^* \geq w_1 \cdot w^* + M\gamma$$

Since w_1 is the zero vector, $w_1 \cdot w^* = 0$, and:

$$w_{M+1} \cdot w^* \geq M\gamma$$

Now, consider $\|w_{t+1}\|^2$:

$$\|w_{t+1}\|^2 = \|w_t + yX\|^2 = \|w_t\|^2 + 2y(w_t \cdot X) + y^2\|X\|^2$$

We know that $y^2 = 1$ (since $y \in \{-1, 1\}$) and that $\|X\|^2 \leq R^2$ because all points lie within a ball of radius R . So:

$$\|w_{t+1}\|^2 \leq \|w_t\|^2 + 2y(w_t \cdot X) + R^2$$

After M mistakes, the norm is bounded:

$$\|w_{M+1}\|^2 \leq MR^2 \implies \|w_{M+1}\| \leq R\sqrt{M}$$

We have:

$$(a) \ w_{M+1} \cdot w^* \geq M\gamma$$

$$(b) \ \|w_{M+1}\| \leq R\sqrt{M}$$

Since w^* is a unit vector, $w_{M+1} \cdot w^* \leq \|w_{M+1}\|$ (by Cauchy-Schwarz), and thus:

$$M\gamma \leq w_{M+1} \cdot w^* \leq \|w_{M+1}\| \leq R\sqrt{M}$$

$$M\gamma \leq R\sqrt{M} \implies \sqrt{M} \leq \frac{R}{\gamma} \implies M \leq \left(\frac{R}{\gamma}\right)^2$$

Thus, the number of mistakes M is bounded above by $\left(\frac{R}{\gamma}\right)^2$

3 Margin Perceptron Algorithm [15 Pts]

Suppose we have a set of examples S and we want to find a large-margin separator for them. One approach is to directly solve for the maximum-margin separator using convex programming (the SVM algorithm). However, if we only need to approximately maximize the margin, then another approach is to use a variation on the Perceptron algorithm called the Margin Perceptron algorithm.

The Margin Perceptron algorithm works just like the standard Perceptron algorithm, except instead of just updating on a mistake it also updates when the current hypothesis gets the example correct but by margin less than $(1 - \epsilon)\gamma$. Assuming that data lies in a ball of radius R and is separable by margin γ , then we can show that this will make at most $O((R/\epsilon\gamma)^2)$ updates. Specifically, the algorithm is as follows

The Margin Perceptron Algorithm (γ)

1. Initialize $t = 1$ and start with the all-zeros weight vector w_1
2. Given an example x , predict positive iff $w_t \cdot x \geq 0$
3. Given the true label y for x , if x was classified incorrectly or classified correctly but by a margin less than $(1 - \epsilon)\gamma$ (that is, $y(w_t \cdot x)/\|w_t\| < (1 - \epsilon)\gamma$), then update the weight vector:
 - (a) Let $w_{t+1} = w_t + yx$. That is, if x is positive then let $w_{t+1} = w_t + x$ and if x is negative then let $w_{t+1} = w_t - x$.
 - (b) Let $t = t + 1$
1. [10 Points] Prove the following theorem regarding the Margin Perceptron Algorithm:

Margin Perceptron Theorem: *Let S be a sequence of labeled examples consistent with a linear separator $w^* \cdot x = 0$ of margin γ . That is, $\|w^*\| = 1$ and*

$$\gamma = \min_{(x,y) \in S} y(w^* \cdot x)$$

Then the number of weight updates made by Margin Perceptron(γ) on S is in $O((R/\epsilon\gamma)^2)$.

(Hint: what happens to $w_t \cdot w^*$ and $\|w_t\|$ every time we make an update?) (Hint: $\sqrt{1 + \alpha} \leq 1 + \frac{\alpha}{2}$)

Please add your solution to the box on the next page.

Let w^* denote the optimal weight vector that separates the data with margin γ . For all correctly classified examples, we have that $w^* \cdot x \geq \gamma$.

The update rule for the margin perceptron is the same as the standard perceptron, e.g., $w_{t+1} = w_t + yx$, where $y \in \{-1, 1\}$ is the label of example x .

Consider $w_{t+1} \cdot w^*$:

$$w_{t+1} \cdot w^* = (w_t + yx) \cdot w^* = w_t \cdot w^* + y(x \cdot w^*)$$

Since the examples are separable with margin γ , then $y(x \cdot w^*) \geq \gamma$. So,

$$w_{t+1} \cdot w^* \geq w_t \cdot w^* + \gamma$$

So, after each update, the dot product $w_t \cdot w^*$ increases by at least γ .

After M updates, we have that the dot product $w_{M+1} \cdot w^*$ has been updated by at least γ by M times:

$$w_{M+1} \cdot w^* \geq M\gamma$$

Now, consider $\|w_{t+1}\|^2$

$$\|w_{t+1}\|^2 = \|w_t + yx\|^2 = \|w_t\|^2 + 2y(w_t \cdot x) + \|x\|^2 = \|w_t\|(\|w_t\| + \frac{2y(w_t \cdot x)}{\|w_t\|} + \frac{\|x\|^2}{\|w_t\|})$$

Since the data lies within a ball of radius R , $\|x\|^2 \leq R^2$, and given $\frac{y(w_t \cdot x)}{\|w_t\|} \leq 2(1 - \epsilon)\gamma$, we have:

$$\|w_{t+1}\|^2 \leq \|w_t\|(\|w_t\| + 2(1 - \epsilon)\gamma + \frac{R^2}{\|w_t\|}) = \|w_t\|^2(1 + \frac{2\gamma(1 - \epsilon)}{\|w_t\|} + \frac{R^2}{\|w_t\|^2})$$

$$\|w_{t+1}\|^2 \leq \|w_t\|^2(1 + \frac{2\gamma(1 - \epsilon)\|w_t\| + R^2}{\|w_t\|^2})$$

Taking the square root, then applying the fact that $\sqrt{1 + \alpha} \leq 1 + \frac{\alpha}{2}$:

$$\|w_{t+1}\| \leq \|w_t\|\sqrt{1 + \frac{2\gamma(1 - \epsilon)\|w_t\| + R^2}{\|w_t\|^2}} \leq \|w_t\|(1 + \frac{2\gamma(1 - \epsilon)\|w_t\| + R^2}{2\|w_t\|^2})$$

$$\|w_{t+1}\| \leq \|w_t\|(1 + \frac{\gamma(1 - \epsilon)}{\|w_t\|} + \frac{R^2}{2\|w_t\|^2}) = \|w_t\| + \gamma(1 - \epsilon) + \frac{R^2}{2\|w_t\|}$$

We know that $\|w_t\|$ is bounded ($\|w_t\| \geq \frac{R^2}{\epsilon\gamma}$), so:

$$\|w_{t+1}\| \leq \|w_t\| + \gamma(1 - \epsilon) + \frac{\epsilon\gamma}{2} = \|w_t\| + \frac{2\gamma - 2\epsilon\gamma + \epsilon\gamma}{2} = \|w_t\| + \frac{(2 - \epsilon)\gamma}{2}$$

After M mistakes, we have that:

$$\|w_{t+1}\| \leq \frac{R^2}{\epsilon\gamma} + \frac{M\gamma(2 - \epsilon)}{2} = \frac{2R^2 + M\epsilon\gamma^2(2 - \epsilon)}{2\epsilon\gamma}$$

Now, by Cauchy-Shwarz, we know that $w_{t+1} \cdot w^* \leq \|w_{t+1}\|\|w^*\|$, and $\|w^*\| = 1$, so:

$$M\gamma \leq w_{t+1} \cdot w^* \leq \|w_{t+1}\| \leq \frac{2R^2 + M\epsilon\gamma^2(2 - \epsilon)}{2\epsilon\gamma} \implies M\gamma \leq \frac{2R^2 + M\epsilon\gamma^2(2 - \epsilon)}{2\epsilon\gamma}$$

Solving for M gives:

$$M \leq \frac{2R^2}{\epsilon^2\gamma^2} = 2\left(\frac{R}{\epsilon\gamma}\right)^2$$

Thus, the number of weight updates made by the margin perceptron is in $O\left(\frac{R}{\epsilon\gamma}\right)^2$

2. **[5 Points]** Suppose you are given a set S of labeled examples up front that are linearly separable by margin γ . How would you use the Margin Perceptron algorithm to find a linear separator for S of margin at least $(1 - \epsilon)\gamma$? (In other words, how could you convert the online algorithm described above into an algorithm you could run on a batch of examples that would produce a separator with large margin over all the examples in the batch?)

Initialize $t = 1$ and start with an all zero weight vector w_1 .

For the batch approach, we can feed the whole set S of labeled examples into the margin perceptron algorithm until convergence.

For each example, repeat:

- (a) predict positive if $w_t \cdot x_i \geq 0$, else negative.
- (b) check the margin condition: if $y_i(w_t \cdot x_i) < (1 - \epsilon)\gamma$, then update the weight vector ($w_{t+1} = w_t + y_i x_i$)

The algorithm should stop either after a maximum number of iterations occurs or the algorithm converges. The resulting final weight vector gives the linear separator for S of margin at least $(1 - \epsilon)\gamma$.

4 Runtime of Perceptron Algorithm [15 Pts]

To explore the worse-case time complexity of the Perceptron algorithm consider the following set of $n - 1$ examples over $\{0, 1\}^n$. Define e_i as the unit vector with a 1 in the i th coordinate and start with two positive examples $(e_1, +)$ and $(e_2, +)$. Then for all $i = 1, \dots, \frac{n}{4} - 1$ add two positive and two negative examples as follows

$$\begin{aligned} &(e_{4i-3} + e_{4i-2} + e_{4i-1}, -), \\ &(e_{4i-3} + e_{4i-2} + e_{4i}, -), \\ &(e_{4i-1} + e_{4i} + e_{4i+1}, +), \\ &(e_{4i-1} + e_{4i} + e_{4i+2}, +) \end{aligned}$$

[15 points] What is the time complexity of the Perceptron algorithm when run on this dataset with an initial weight of all 0s. Specifically, we repeatedly cycle the algorithm through examples until $w \cdot x > 0$ for all positive examples $x \in S$ and $w \cdot x < 0$ for all negative examples $x \in S$. For simplicity use the version that does not normalize examples (i.e. when a mistake is made on a positive example x , it sets $w = w + x$, rather than $w = w + x/\|x\|$, and similarly for mistakes on negatives). This won't really matter since $\|x\| \leq \sqrt{n}$ for $x \in \{0, 1\}^n$, but it is easier to think about since the w_i will always be integral.

The dataset follows a pattern:

X_1	X_2	X_3	X_4	...	X_n	Y
1	0	0	0	0	...	+
0	1	0	0	0	...	+
1	1	1	0	0	...	-
1	1	0	1	0	...	-
0	0	1	1	1	...	+
0	0	1	1	0	...	+
0	0	0	0	1	...	-
0	0	0	0	1	...	-

The weight vector is initialized as the zero vector. Each time a mistake is made, the weight vector will be updated, e.g., $w_{t+1} = w_t + y_t x_t$.

The examples alternate between two positive and two negative. The pattern of the data suggests that many mistakes will occur before the algorithm converges. Before, we have seen that the perceptron makes at most $O(\frac{R^2}{\gamma^2})$ mistakes, with R being the maximum norm of any input vector (in this case $\sqrt{n}$), and γ being the margin of the dataset. For this specific dataset, we can conclude that γ is quite small, due to the close clustering of positive and negative examples in the dataset. Since our dataset $\in \{0, 1\}^n$, then the number of updates needed will be at worst $O(n)$.

Combined with the operation of updating the weight vector, in which we will need to iterate through n positions, this operation is also $O(n)$. Thus, the final time complexity of the perceptron algorithm on this dataset is approximately $O(n \cdot n) = O(n^2)$.

5 Collaboration Questions

1. (a) Did you receive any help whatsoever from anyone in solving this assignment? **Solution Yes**
(b) If you answered ‘yes’, give full details (e.g. “Jane Doe explained to me what is asked in Question 3.4”)

Solution

Maddie Shaklee confirmed my understanding of question 1.6 re: depth of tree & node splitting
Ellie Escobar-Robledo explained to me the premise of the dataset given in question 4.

2. (a) Did you give any help whatsoever to anyone in solving this assignment? **Solution Yes**
(b) If you answered ‘yes’, give full details (e.g. “I pointed Joe Smith to section 2.3 since he didn’t know how to proceed with Question 2”)

Solution

I explained the concept of question 3.1 to Ellie Escobar-Robledo.

3. (a) Did you find or come across code that implements any part of this assignment? **Solution No.**
(b) If you answered ‘yes’, give full details (book & page, URL & location within the page, etc.).

Solution